

Dynamic Programming in 2D



DNA Comparison:

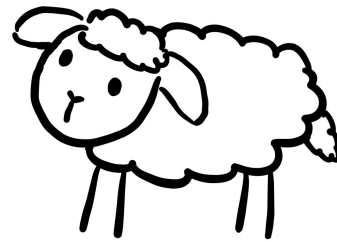
Longest Common Subsequence (LCS)

The length of a longest common subsequence (LCS) between two genomes is a measure of similarity.

S₁ (length N) ACCGGTCGAGTGCGCGGAAGCCGGCCGAA

S₂ (length M) GTCGTTCGGAATGCCGTTGCTCTGTAA

What is an LCS of
“Go Blue” and “Ghoul”?



The DP Recipe

1. Define *value version* of what we want to compute
2. Derive a recurrence *“Let $OTV(J_1, \dots, J_i)$ be...”*
3. Size of table: How many dimensions? Range of each dimension?
4. What are the base case(s)?
5. To fill in a cell, which other cells need to be filled already? In which order do I fill the table?
6. Which cell(s) contain the final answer?
7. Running time is often (size of table) $\cdot$ (time to fill each entry)
8. To reconstruct the solution (instead of just its value) follow arrows from final answer to base case

LCS Recurrence

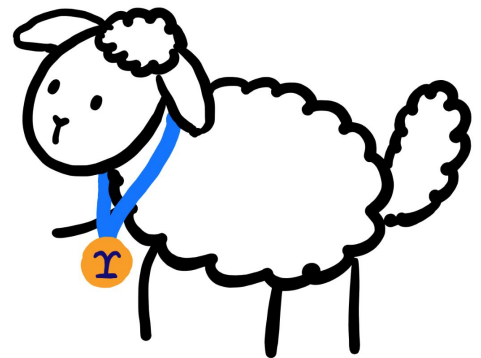
Let $\text{LCS}(S1[1..i], S2[1..j])$ be the *length* of the LCS of $S1[1..i], S2[1..j]$.

Here's my recurrence! I tested it on 1 example and it worked!

$\text{LCS}(S1[1..i], S2[1..j]) =$

$$\max \begin{cases} \text{LCS}(S1[1..i-1], S2[1..j-1]) + 1 & \text{(use it! i.e. match } S1[i], S2[j]) \\ \text{LCS}(S1[1..i-1], S2[1..j-1]) & \text{(lose it!)} \end{cases}$$

Counterexample:



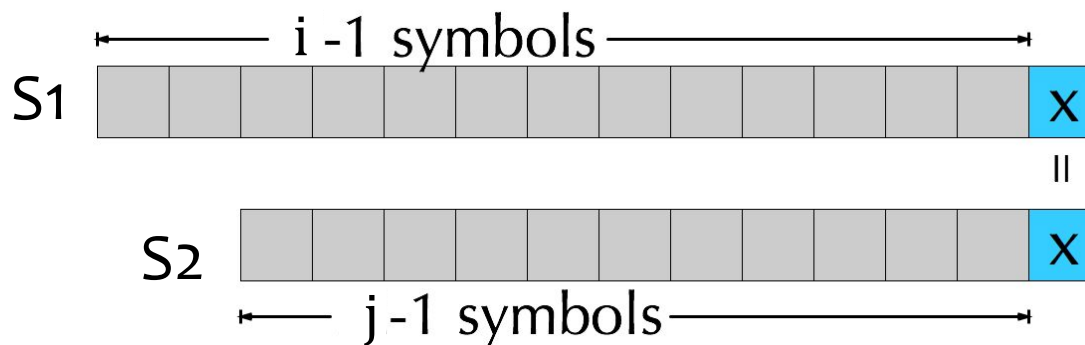
LCS Recurrence

Let $\text{LCS}(S1[1..i], S2[1..j])$ be the *length* of the LCS of $S1[1..i], S2[1..j]$.

Case 1: Suppose $S1[i] = S2[j]$.

Claim. There exists an optimal solution for $S1[1..i], S2[1..j]$ that matches $S1[i]$ and $S2[j]$.

Proof.



Case 1 recurrence: $\text{LCS}(S1[1..i], S2[1..j]) = \text{LCS}(S1[1..i-1], S2[1..j-1]) + 1$

LCS Recurrence

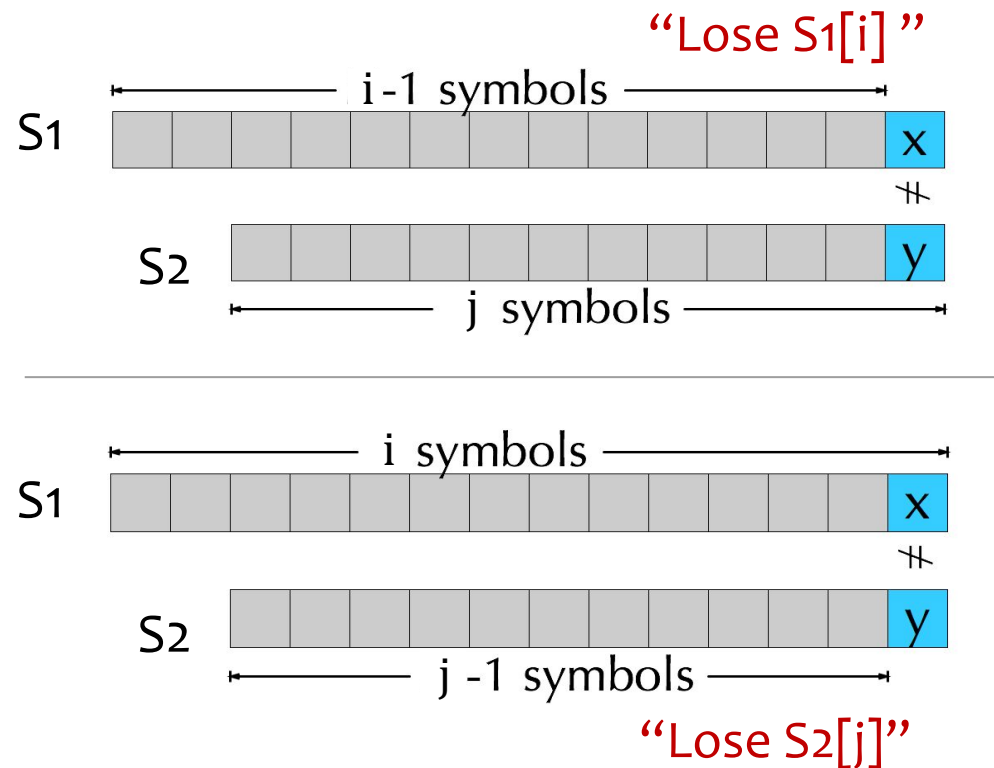
Case 2: Suppose $S1[i] \neq S2[j]$

Optimal solution for $S1[1..i], S2[1..j]$ can't have both $S1[i]$ and $S2[j]$.

⇒ **“lose it or lose it!”**

Case 2 recurrence:

$LCS(S1[1..i], S2[1..j]) =$



Full Recurrence for LCS

$\text{LCS}(S1[1..i] , S2[1..j]) =$

$\left\{ \right.$

if $S1[i] = S2[j]$

otherwise

Base cases:

Let's Follow the DP Recipe

$S_1 = \text{GAC}; S_2 = \text{AGCAT}; \text{table}(i,j) = \text{LCS}(S_1[1..i], S_2[1..j])$

	$\emptyset$	A	G	C	A	T
$\emptyset$						
G						
A						
C						

Running time:

A visualization: <https://www.cs.usfca.edu/~galles/visualization/DPLCS.html>

Pseudocode

Algorithm LCS($S_1[1..N]$, $S_2[1..M]$)

table := 2D-array indexed from 0 to N, and 0 to M

for $i = 0$ to N: **table**($i, 0$) = 0

for $i = 0$ to M: **table**(0, i) = 0

for $i = 1$ to N:

 for $j = 1$ to M:

 if $S_1[i] = S_2[j]$:

table(i, j) = 1 + **table**($i-1, j-1$)

 else:

table(i, j) = $\max\{\text{table}(i-1, j), \text{table}(i, j-1)\}$

Return table(N,M)

Structure of **all** DP pseudocode:
loop through table in appropriate
order and fill in cells according to
recurrence.

Can we do better than $O(MN)$ time?

Under a hypothesis called the “strong exponential time hypothesis”, there is no algorithm even in time $O(MN^{0.99})$! [Abboud, Backurs, Vassilevska Williams ‘15]

If the hypothesis is true, you’ve seen the (essentially) optimal algorithm.

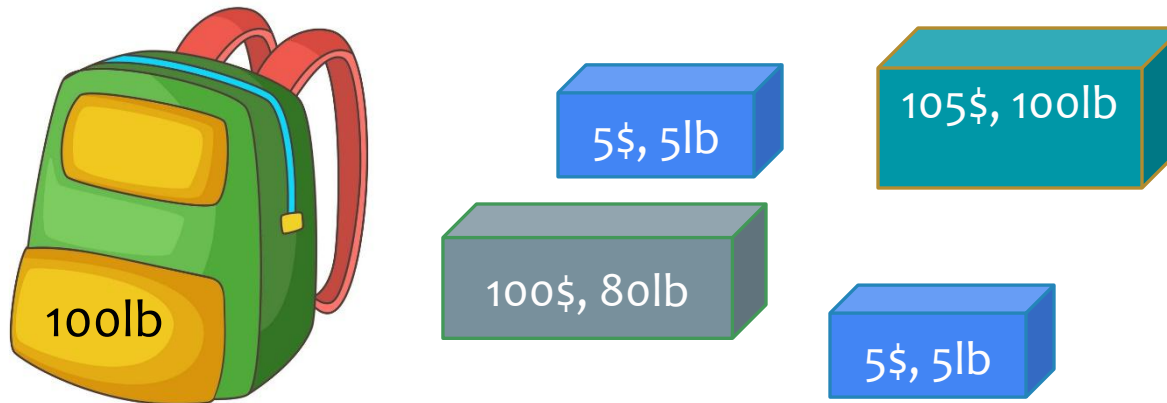
But... $|\text{human genome}| \approx 3\text{bil}$, $|\text{chimp genome}| \approx 2.8\text{bil}$

– **Current research:** faster approximation algorithms

Knapsack Problem

Input: A set of n items $t_1, \dots, t_n$, each with an integer value v_i and weight w_i , along with an integer C = total “weight capacity” of the knapsack.

Output: A set of items whose total weight is at most C and whose total value is maximized.



Question: What is the input size in terms of n and C ?

(Input size note: suppose each individual v_i is small enough that we can disregard its contribution to the input size, but each w_i could be up to C)

Knapsack Recurrence

Let $\text{Knapsack}(\{t_1, \dots, t_i\}, c)$ be the optimal value attainable with items $\{t_1, \dots, t_i\}$ and weight capacity c .

Consider item t_i .

$$\text{Knapsack}(\{t_1, \dots, t_i\}, c) = \max \left\{ \begin{array}{l} \text{(use it!)} \\ \text{(lose it!)} \end{array} \right.$$

Base cases:

Let's follow the DP Recipe

weight limit c

items up through..

	$\emptyset$	1	2	3
$\emptyset$				
t ₁ : 4\$, 2lb				
t ₂ : 8\$, 1lb				
t ₃ : 6\$, 2lb				
t ₄ : 5\$, 1lb				

Running time:

Pseudocode

Algorithm Knapsack($\{t_1, \dots, t_n\}$, C)

table := 2D-array indexed from 0 to n , and 0 to C

for $j = 0$ to C : **table**(0, j) = 0

for $i = 1$ to n :

 for $j = 1$ to C :

 if $w_i \leq j$:

table(i, j) = $\max(v_i + \text{table}(i-1, j - w_i), \text{table}(i-1, j))$

 else:

table(i, j) = **table**($i-1, j$)

Return table(n, C)

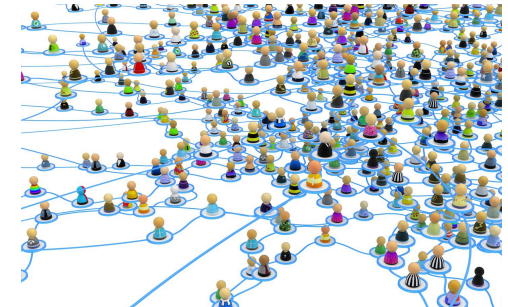
Structure of **all** DP pseudocode:
loop through table in appropriate
order and fill in cells according to
recurrence.

Next Time: DP on Graphs

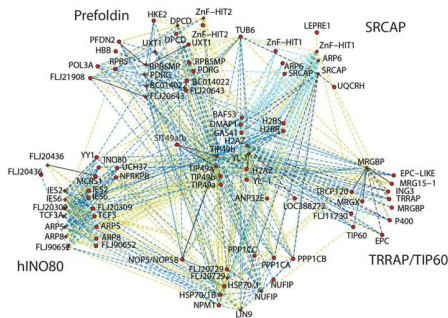
Graphs can represent many things!



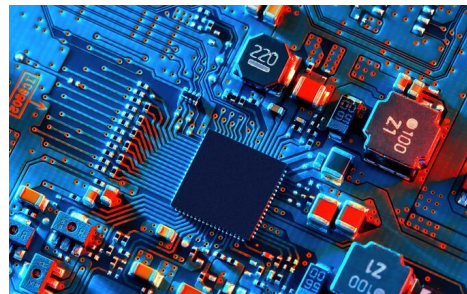
The Internet



Social Networks



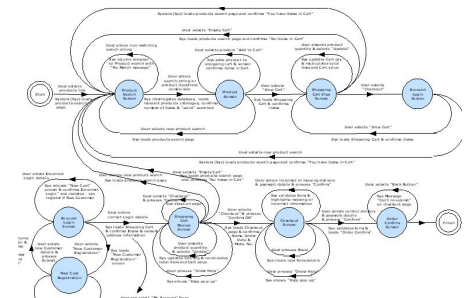
Biological and Chemical Networks



Circuits



Transportation Networks



State Transition Networks